

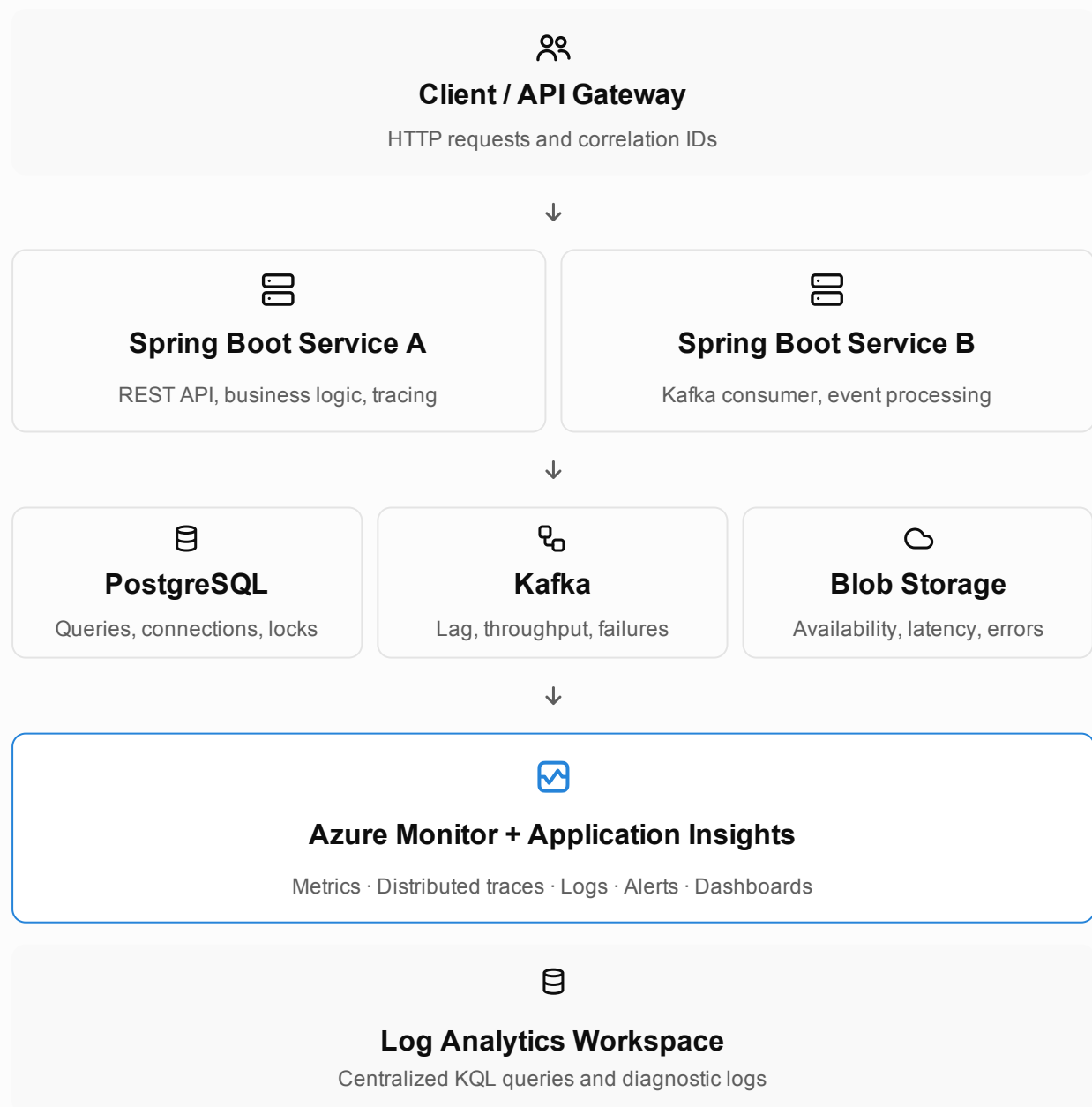
Azure Monitor for Spring Boot Microservices, Kafka, PostgreSQL and Azure Blob Storage

This walkthrough explains how to implement **end-to-end monitoring and troubleshooting** for a Spring Boot microservices application running on Azure, with Kafka event processing, PostgreSQL, and Blob Storage.

We'll use an enterprise-style architecture similar to a production microservices environment.

1. What you will monitor

Monitoring architecture



Monitoring responsibilities

Component	What to monitor
Spring Boot	Request rate, failures, p95/p99 latency, JVM, GC, exceptions
Kafka	Consumer lag, throughput, failed messages, rebalance events
PostgreSQL	CPU, storage, connections, slow queries, locks
Blob Storage	Availability, latency, throttling, failed operations
Azure infrastructure	CPU, memory where supported, restarts, availability
End-to-end	Trace ID, request flow, dependency failures

2. Step 1 — Create Azure Monitor resources

Azure Portal navigation

Screen 1 — Azure Portal

Portal search

 Search resources, services, and docs

Search for: `Application Insights`

Create Application Insights

1. Click **Create**.
2. Select your Subscription and Resource Group.
3. Enter Name: `ai-microservices-prod`.
4. Choose Region matching your application deployment where possible.
5. Create or select a Log Analytics Workspace: `law-microservices-prod`.
6. Click **Review + Create** → **Create**.

Expected result: Application Insights resource created

Application Insights collects application telemetry, while Log Analytics provides a workspace for querying collected logs.

Recommended resource naming

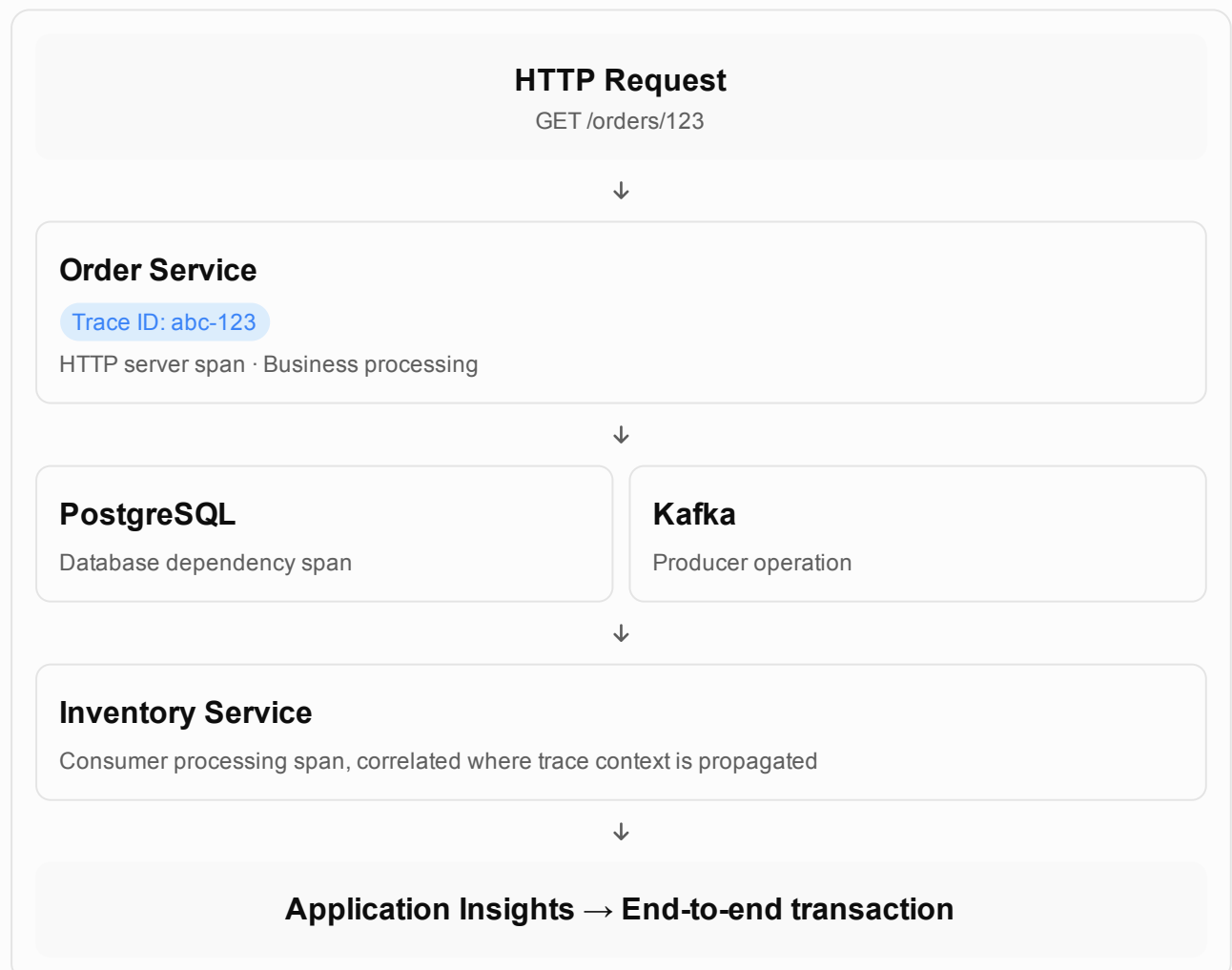
Resource	Example
Resource Group	<code>rg-microservices-prod</code>
Application Insights	<code>ai-microservices-prod</code>
Log Analytics Workspace	<code>law-microservices-prod</code>
Action Group	<code>ag-microservices-oncall</code>
Dashboard	<code>dash-microservices-health</code>

For production, use separate Application Insights resources for production and non-production environments.

3. Step 2 — Instrument Spring Boot microservices

This is the most important step because it enables distributed tracing across services.

What happens after instrumentation?



Option A: Java agent (recommended for an existing Spring Boot application)

The Java agent provides automatic instrumentation without requiring you to add instrumentation code throughout your application.



A1. Download the Application Insights Java agent

Download the current Java agent from the official documentation:

[Microsoft Learn — Enable OpenTelemetry in Application Insights for Java ↗](#)

Place the agent in your application deployment directory.

Example:

```
/app
├─ app.jar
└─ applicationinsights-agent.jar
```

A2. Configure the connection string

In Azure Portal:

Screen 2 — Application Insights Overview

 ai-microservices-prod

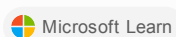
Overview

Connection string

InstrumentationKey=...;IngestionEndpoint=...

Copy

Copy the connection string from the Overview page. Use a secure application setting or Key Vault-backed configuration rather than committing it to source control.



A3. Add Java agent to Docker

dockerfile

```
FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/order-service.jar app.jar
COPY applicationinsights-agent.jar .

ENV APPLICATIONINSIGHTS_CONNECTION_STRING=""

ENV OTEL_SERVICE_NAME="order-service"

ENTRYPOINT ["java", \
    "-javaagent:/app/applicationinsights-agent.jar", \
    "-jar", \
    "/app/app.jar"]
```

Pass the connection string through your deployment environment.

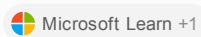
For example, in Kubernetes:

YAML

```
env:
  - name: APPLICATIONINSIGHTS_CONNECTION_STRING
    valueFrom:
      secretKeyRef:
        name: app-insights-secret
        key: connection-string

  - name: OTEL_SERVICE_NAME
    value: order-service
```

The Java agent is enabled using the `-javaagent` JVM argument, and the connection string can be configured using `APPLICATIONINSIGHTS_CONNECTION_STRING`.



Repeat for:

- `order-service`
- `inventory-service`
- `payment-service`
- `notification-service`

Use a unique service name for each service.

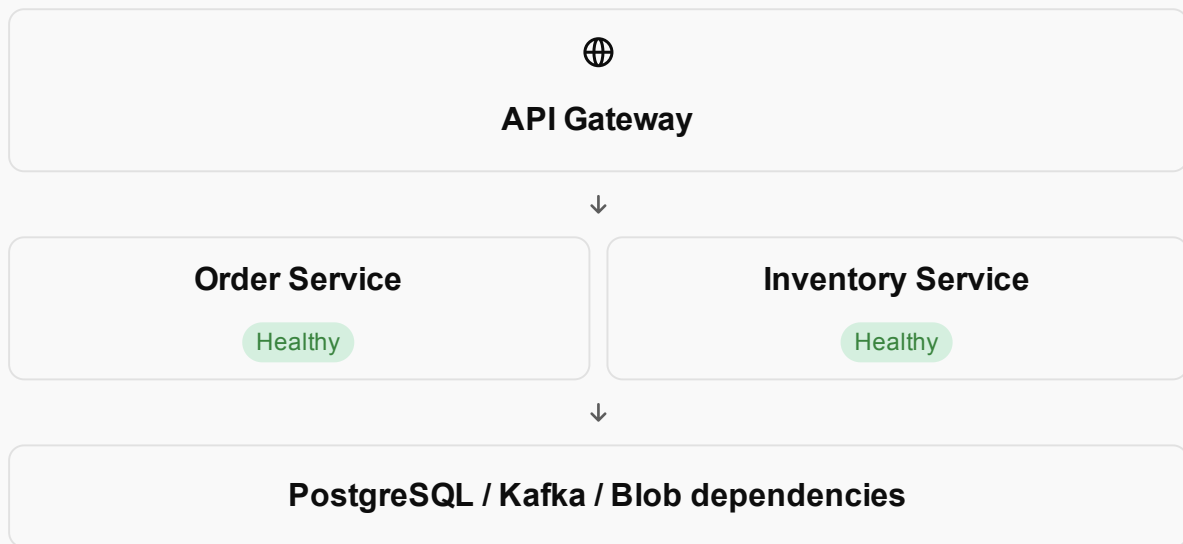
A4. Verify telemetry

Start your Spring Boot application and generate a few HTTP requests.

In Azure Portal:

Application Insights → Investigate → Application Map

Screen 3 — Application Map (expected layout)



Illustrative view of the service relationships, not a captured Azure Portal screenshot.

If the application map is empty, check the connection string, agent startup logs, outbound connectivity, and whether requests have actually reached the application.

4. Step 3 — Monitor Kafka events

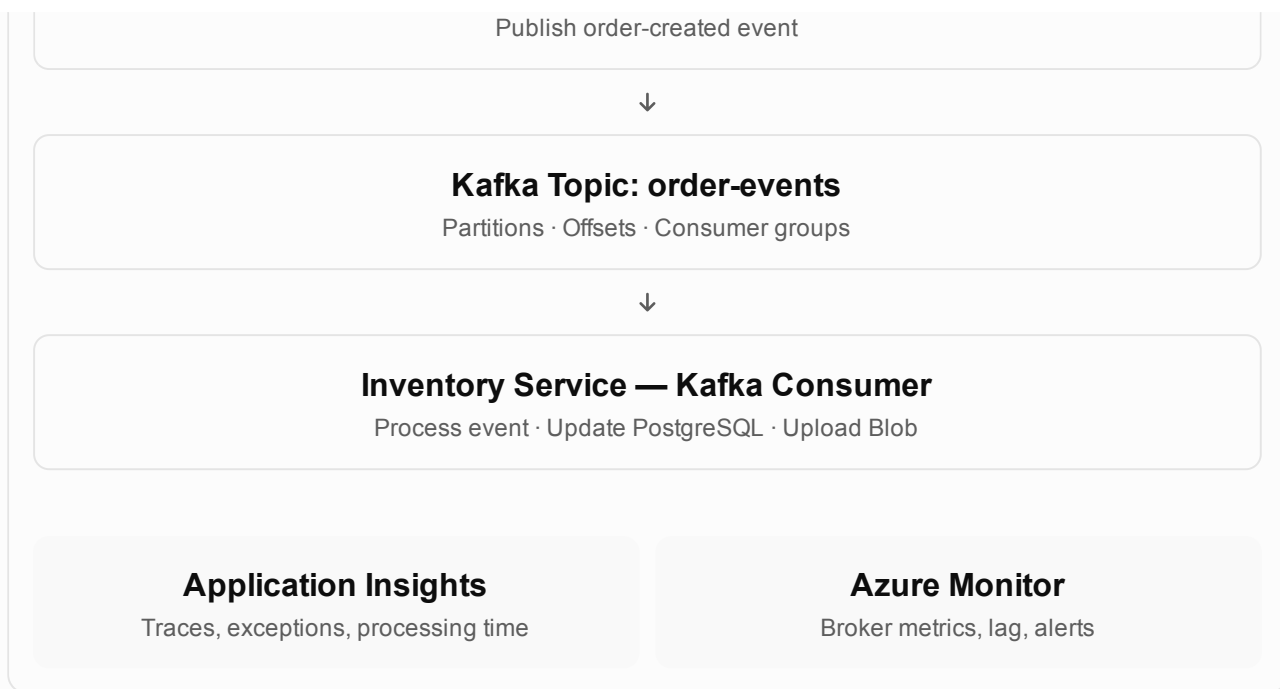
Kafka monitoring requires two complementary approaches:

1. Application Insights for producer/consumer operations, exceptions and distributed tracing.
2. Kafka broker/client metrics for throughput, consumer lag, partition health and rebalances.

Application Insights does not automatically provide every Kafka broker metric.

Kafka monitoring architecture

Order Service — Kafka Producer



A. Instrument Kafka consumer processing

Example using Spring Kafka:

Java

```
@Component
public class OrderEventConsumer {

    private final MeterRegistry meterRegistry;

    public OrderEventConsumer(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
    }

    @KafkaListener(
        topics = "order-events",
        groupId = "inventory-service"
    )
    public void consume(OrderEvent event) {

        Timer.Sample sample =
            Timer.start(meterRegistry);

        try {
            processOrder(event);

            meterRegistry.counter(
                "kafka.events.processed",
                "topic", "order-events",
                "status", "success"
            ).increment();
        }
```

```

    } catch (Exception ex) {

        meterRegistry.counter(
            "kafka.events.processed",
            "topic", "order-events",
            "status", "failure"
        ).increment();

        throw ex;

    } finally {
        sample.stop(
            Timer.builder("kafka.event.processing")
                .tag("topic", "order-events")
                .register(meterRegistry)
        );
    }
}

private void processOrder(OrderEvent event) {
    // Business processing
}
}

```

This records event processing count and duration through Micrometer. Configure an appropriate Azure Monitor exporter or OpenTelemetry metrics pipeline to export these metrics.

Avoid putting order IDs, customer IDs, or other high-cardinality values into metric dimensions.

B. Kafka metrics to monitor

Metric	What it tells you
Consumer lag	How far consumers are behind
Messages consumed/sec	Consumer throughput
Processing duration	Time to process each event
Failed events	Processing failures
Rebalances	Consumer group instability
Retry count	Transient failure frequency
Dead-letter count	Events requiring investigation

For consumer lag, use broker/client monitoring or your Kafka platform's metrics integration. For Azure-hosted Kafka-compatible services, use the service's native Azure Monitor metrics where available.

C. Example alert thresholds

These are starting operational thresholds, not universal limits.

Alert	Example condition
High consumer lag	Lag above 1,000 for 5 minutes
Processing failures	Failure rate above 2%
Slow event processing	p95 above 2 seconds
Dead-letter messages	Any unexpected increase
Consumer unavailable	No successful processing for 5 minutes

Tune thresholds based on topic volume, business SLA, and normal traffic.

5. Step 4 — Monitor PostgreSQL

Assumption: You are using **Azure Database for PostgreSQL Flexible Server**.

A. Enable diagnostic settings

Screen 4 — PostgreSQL Diagnostic Settings

PostgreSQL Flexible Server

Left navigation



Monitoring



Diagnostic settings

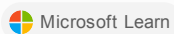
Select

Configure the setting

1. Click **Add diagnostic setting**.
2. Name it `diag-postgres-prod`.
3. Select the PostgreSQL server logs and relevant Query Store/session categories.

4. Select **AllMetrics** if you want metrics exported to the workspace.
5. Under Destination details, select **Send to Log Analytics workspace**.
6. Select `law-microservices-prod`.
7. Choose **Resource specific** destination tables.
8. Click **Save**.

Microsoft documents these diagnostic settings and PostgreSQL log categories, including server logs, sessions, Query Store runtime and wait statistics.



Official documentation:

[Configure and access PostgreSQL logs ↗](#)

B. PostgreSQL metrics

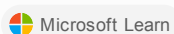
In the PostgreSQL resource:

`Monitoring → Metrics`

Track:

- CPU percentage
- Memory percentage
- Active connections
- Storage percentage
- Read/write throughput
- Disk IOPS
- Transaction log storage

Azure PostgreSQL exposes platform metrics, generally at one-minute intervals, with some metrics processed in batches.



C. Query PostgreSQL logs using KQL

Navigate to:

`Azure Portal → Log Analytics Workspace → Logs`

Run:

kusto

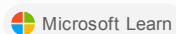
```
PGSQLServerLogs
| where TimeGenerated > ago(1h)
| order by TimeGenerated desc
| take 100
```

Find errors:

kusto

```
PGSQLServerLogs
| where TimeGenerated > ago(24h)
| where Message has_any (
    "ERROR",
    "FATAL",
    "deadlock",
    "timeout"
)
| project TimeGenerated,
    LogicalServerName,
    Message
| order by TimeGenerated desc
```

For query performance, inspect the Query Store runtime and wait-statistics data after enabling those categories.



6. Step 5 — Monitor Azure Blob Storage

A. Enable Blob diagnostics

Screen 5 — Storage Account

Storage Account

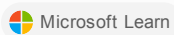
Left navigation



Monitoring

1. Open your Storage Account.
2. Select the Blob service resource if required.
3. Navigate to Diagnostic settings.
4. Click Add diagnostic setting.
5. Enable relevant read, write and delete logs, or the supported all-logs category group.
6. Enable AllMetrics.
7. Send logs to `law-microservices-prod`.
8. Save.

Azure diagnostic settings route selected resource logs and metrics to destinations such as Log Analytics.



B. Blob metrics

Navigate to:

`Storage Account → Monitoring → Metrics`

Monitor:

- Availability
- Success E2E latency
- Success server latency
- Transactions
- Ingress / egress
- Throttling and failed requests

C. Blob operation logs

After enabling diagnostic logs, query the `StorageBlobLogs` table.

Find failed Blob operations:

```
kusto
```

```
StorageBlobLogs
| where TimeGenerated > ago(24h)
| where StatusText !contains "Success"
| project TimeGenerated,
           OperationName,
           StatusCode,
           StatusText,
           Uri
| order by TimeGenerated desc
```

Find slow Blob operations:

kusto

```
StorageBlobLogs
| where TimeGenerated > ago(24h)
| top 20 by DurationMs desc
| project TimeGenerated,
           OperationName,
           DurationMs,
           ServerLatencyMs,
           Uri
```

These queries use the documented Blob Storage log schema.

 Microsoft Learn +1

7. Step 6 — Build an end-to-end troubleshooting dashboard

Create a dashboard showing the application and its dependencies.

Recommended dashboard layout

Microservices Production Health

HTTP Request Failure Rate

0.4%

Example

API p99 Latency

420 ms

Example

Kafka Consumer Lag

120

Example

PostgreSQL CPU

58%

Example

Service health

✔ Order Service	Healthy
✔ Inventory Service	Healthy
⚠ Kafka Consumer	Lag elevated
✔ PostgreSQL	Healthy

Illustrative dashboard values only—not live measurements from your Azure subscription.

Add alerts

Navigate to:

Azure Monitor → Alerts → Create → Alert rule

Configure alerts for:

Alert	Example condition	Action
API failure rate	5xx rate above 5%	Notify on-call
API latency	p99 above 1 second	Investigate
PostgreSQL CPU	Above 85% for 10 min	Notify DBA
PostgreSQL storage	Above 85%	Capacity review
Kafka lag	Above agreed threshold	Notify service owner
Blob failures	Unexpected increase in failed operations	Investigate storage access

Use an Action Group to send notifications to email, SMS, or supported incident-management integrations.

8. Step 7 — End-to-end production troubleshooting example

Consider this scenario:

A customer submits an order, but the inventory update is delayed.

Follow this sequence:

Incident investigation workflow

1 Check Application Insights

Search the failed or slow request in Transaction Search. Capture its operation/trace ID.

2 Inspect the distributed trace

Identify whether time was spent in the API, database call, Kafka publishing, or another dependency.

3 Check Kafka consumer lag

Determine whether the event was produced but is waiting for consumption.

4 Inspect consumer exceptions

Look for deserialization failures, retries, database exceptions, and dead-letter routing.

5 Check PostgreSQL

Review active connections, CPU, slow queries, locks, and Query Store wait statistics.

6 Check Blob Storage

Review failed uploads, authorization errors, throttling, and operation latency.

7 Correlate all evidence

Use timestamps, trace IDs, Kafka event identifiers, and service logs to identify the failing dependency.

For asynchronous Kafka processing, propagate trace context through Kafka headers and ensure your consumer instrumentation extracts that context. A trace ID alone does not guarantee that the producer and consumer will appear in one continuous trace.

9. Screenshot and official reference guide

The portal panels above are illustrative screen-by-screen guides, not captured screenshots from a live Azure subscription. Azure's exact layout can vary by resource type, portal updates, and permissions.

For actual Azure Portal screenshots, these Microsoft Learn pages include visual walkthroughs:



1. Application Insights for Java

Java agent setup, connection string and telemetry verification.

[Open official setup guide ↗](#)



2. PostgreSQL diagnostic settings

Portal configuration and Log Analytics queries.

[Open PostgreSQL monitoring guide ↗](#)



3. Azure Monitor diagnostic settings

Portal screenshots and destination configuration.

[Open diagnostic settings guide ↗](#)



4. Blob Storage monitoring queries

Failed operations, latency and throttling queries.

[Open Blob Storage query examples ↗](#)

Recommended implementation order

For your Spring Boot + Kafka + PostgreSQL + Blob Storage stack, implement in this order:

1. Create Application Insights and Log Analytics.
2. Instrument every Spring Boot service.
3. Verify distributed tracing and service maps.
4. Enable PostgreSQL metrics and diagnostic logs.
5. Enable Blob Storage metrics and diagnostic logs.
6. Instrument Kafka consumer processing and monitor consumer lag.
7. Create production alerts and dashboards.
8. Test the setup by deliberately generating a controlled application error, Kafka processing failure, and Blob Storage failure in a non-production environment.

Key principle: Application Insights gives you the application execution story; Azure Monitor gives you the infrastructure and platform health story; Log Analytics lets you correlate their logs and investigate incidents.